1. Aim

To understand Python functions, parameters, return values and variable scope, and to design a canteen order system that creates an independent order for each customer.

2. Problem Explanation

The canteen program uses a function add_item(item, order=[]) to add food items to a customer's order. This causes a problem because a mutable default list is created only once when the function is defined. Later function calls may reuse the same list, so items from one customer can appear in another customer's order.

3. Demonstration of the Default Parameter Bug

```
def add_item_buggy(item, order=[]):
    order.append(item)
    return order

print(add_item_buggy("Tea"))
print(add_item_buggy("Vada"))
```

Expected output:

```
['Tea']
['Tea', 'Vada']
```

Both calls use the same default list because the default value is created once when the function is defined. The second call therefore sees the item added by the first call. This is called the mutable default argument problem.

4. Correct Function Using None

The safe approach is to use None as the default value. Inside the function, a new empty list is created whenever no order list is supplied.

```
def add_item(item, order=None):
    if order is None:
        order = []
    order.append(item)
    return order

print(add_item("Tea"))
print(add_item("Vada"))
```

Expected output:

```
['Tea']
['Vada']
```

5. Complete the Code – TODO 1

The completed function is:

```
def add_item(item, order=None):
    if order is None:
        order = []
```

```
    order.append(item)
    return order
```

This creates a new list for each independent customer order. If an existing list is passed explicitly, the function appends to that supplied list.

6. Test with Three Customers

```
customer1 = add_item("Tea")
customer1 = add_item("Vada", customer1)

customer2 = add_item("Coffee")
customer2 = add_item("Samosa", customer2)

customer3 = add_item("Juice")

print("Customer 1:", customer1)
print("Customer 2:", customer2)
print("Customer 3:", customer3)
```

Expected output:

```
Customer 1: ['Tea', 'Vada']
Customer 2: ['Coffee', 'Samosa']
Customer 3: ['Juice']
```

Each customer has a separate list, so one customer's items do not appear in another customer's order.

7. Positional, Keyword and Default Parameters

Parameter type	Meaning	Example
Positional	Arguments are assigned according to their order.	add_item("Tea", order_list)
Keyword	Arguments are assigned using parameter names.	add_item(item="Tea", order=order_list)
Default	A value is used when the caller does not supply that argument.	def add_item(item, order=None):

Positional arguments are useful for short, obvious calls. Keyword arguments improve readability when a function has several parameters. Default parameters make common arguments optional.

8. Returning More Than One Value

A Python function can return more than one value by separating the values with commas. Python returns them together as a tuple.

```
def calculate_bill(amount):
    gst = amount * 0.05
    total = amount + gst
    return total, gst

total, gst = calculate_bill(200)

print("GST:", gst)
print("Total:", total)
```

Expected output:

```
GST: 10.0
Total: 210.0
```

9. Local, Global and Nonlocal Scope

Local variable example:

```
def show_local():
    message = "Local variable"
    print(message)

show_local()
```

A local variable is created inside a function and is normally accessible only within that function.

Global variable example:

```
count = 10

def show_global():
    print(count)

show_global()
```

A global variable is defined outside functions. It can be read inside a function. To reassign it inside a function, the global keyword is required.

```
count = 10

def update_global():
    global count
    count = count + 1

update_global()
print(count)  # 11
```

Changing global variables is usually avoided because many functions can modify the same value, making programs harder to test and debug. Passing values as arguments and returning results is usually clearer.

Nonlocal variable example:

```
def outer():
    total = 0

    def inner():
        nonlocal total
        total += 5

    inner()
    print(total)

outer()
```

The nonlocal keyword allows an inner function to reassign a variable from the nearest enclosing function scope. It does not refer to a global variable.

10. Inspecting the Buggy Function Default

Python stores a function's default parameter values in its __defaults__ attribute. After calling the buggy function with no order argument, the shared list can be inspected.

```
def add_item_buggy(item, order=[]):
    order.append(item)
    return order

add_item_buggy("Tea")
add_item_buggy("Vada")

print(add_item_buggy.__defaults__)
```

Expected output:

```
(['Tea', 'Vada'],)
```

The output is a tuple containing the default list. It shows that the same list has been modified by both calls. The assignment wording may refer to add_item_buggy.defaults, but the correct Python attribute is add_item_buggy.__defaults__ (two underscores on each side).

11. Alternative Concept – Explicit List or Closure

One alternative is to require the caller to create and pass a new list for each customer.

```
def add_item_explicit(item, order):
    order.append(item)
    return order

customer_order = []
add_item_explicit("Tea", customer_order)
add_item_explicit("Vada", customer_order)
print(customer_order)
```

Another alternative is a closure. The outer function creates a private order list, and the returned inner function adds items to that list.

```
def make_order():
    order = []

    def add(item):
        order.append(item)
        return order

    return add

customer1_add = make_order()
customer2_add = make_order()

print(customer1_add("Tea"))
print(customer2_add("Coffee"))
```

Each call to make_order() creates a separate list. The inner function remembers the list through its enclosing scope.

12. Time and Space Complexity Analysis

Design	Time per added item	Space per customer	Explanation
None default parameter	O(1) amortised	O(m)	Creates a separate list and appends each item.
Explicitly passed list	O(1) amortised	O(m)	Caller creates and owns a separate list for each customer.
Closure	O(1) amortised	O(m)	Each closure retains its own

			order list.

Here, m is the number of items in one customer's order. Appending to a Python list takes O(1) amortised time. Each design stores the customer's items, so space grows as O(m).

13. Execution Evidence

Run the buggy and corrected functions and capture their outputs. Also run the three-customer test and the __defaults__ inspection to show that the corrected design keeps orders independent.

- Screenshot of the buggy function showing the shared list.
- Screenshot of the corrected function showing separate lists.
- Screenshot of the three-customer test.
- Screenshot showing add_item_buggy.__defaults__.

14. Conclusion

The mutable default argument problem occurs because a default list is created only once when the function is defined. Using None and creating a new list inside the function prevents customers from sharing the same order. Positional, keyword and default parameters, multiple return values, and local, global and nonlocal scope were also demonstrated. The None default design is simple, readable and suitable for the canteen order system.